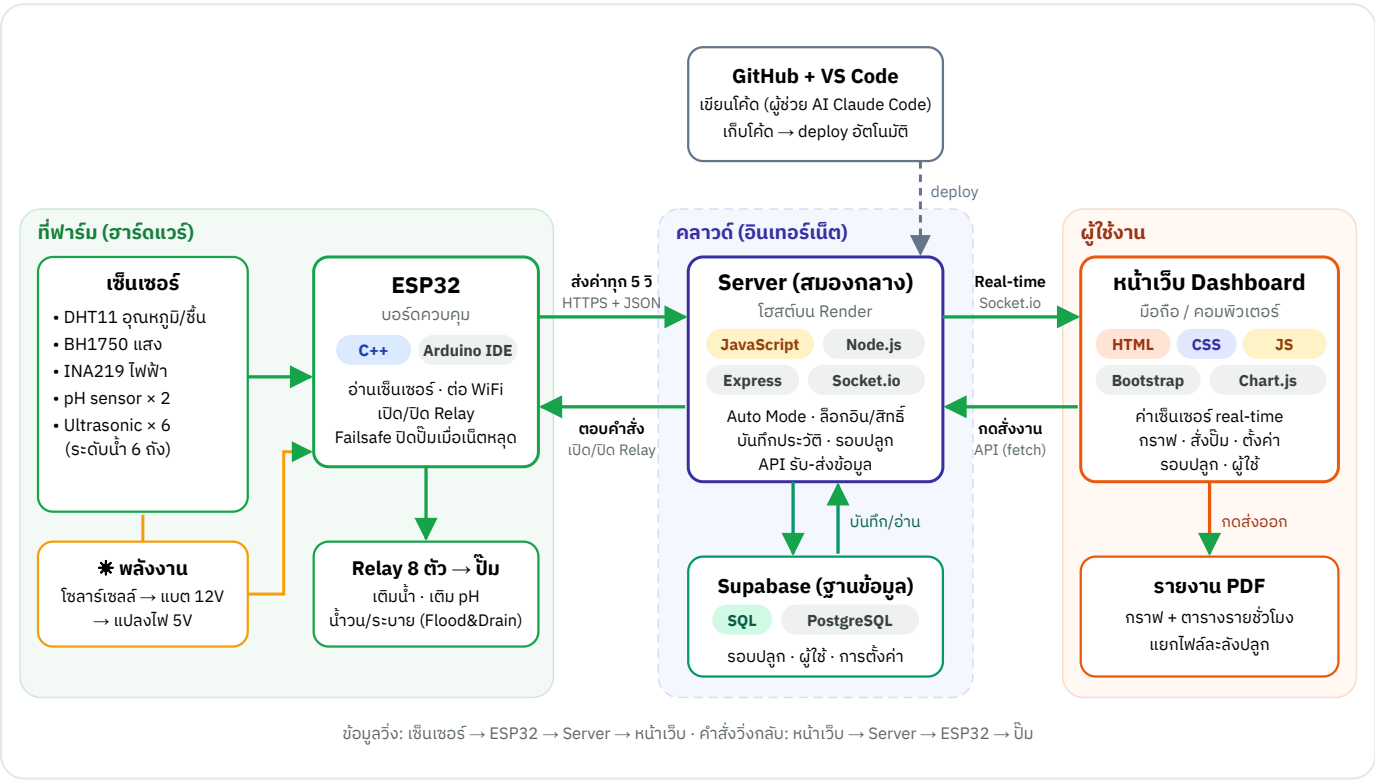
Smart Farm — ระบบปลูกผักไฮโดรโปนิกส์อัตโนมัติ

เว็บนี้สร้างอย่างไร · ใช้ภาษา/โปรแกรมอะไร · ทำอะไรได้บ้าง

ระบบมี 3 ส่วนทำงานร่วมกัน: **ESP32** อ่านเซ็นเซอร์และสั่งปั๊มที่ฟาร์ม → **Server บนคลาวด์** รับข้อมูล ตัดสินใจอัตโนมัติ และเก็บข้อมูล → **หน้าเว็บ** แสดงผลแบบ real-time ใช้สั่งงานได้จากมือถือหรือคอมพิวเตอร์

แผนผังระบบ (System Map)

สีป้ายบอกภาษา: C++ JavaScript HTML CSS SQL โปรแกรม/บริการ



ข้อมูลวิ่ง: เซ็นเซอร์ → ESP32 → Server → หน้าเว็บ · คำสั่งวิ่งกลับ: หน้าเว็บ → Server → ESP32 → ปั๊ม

แต่ละส่วนใช้ภาษาและโปรแกรมอะไร

ส่วน	ภาษา	โปรแกรม / ไลบรารี	หน้าที่
บอร์ด ESP32	C++	Arduino IDE · WiFi · HTTPClient · ArduinoJson · DHT · BH1750 · Adafruit INA219	อ่านเซ็นเซอร์ทุกตัว แพ็กข้อมูลเป็น JSON ส่งขึ้น Server ทุก 5 วินาที แล้วเปิด/ปิด Relay ตามคำสั่งที่ตอบกลับมา
Server	JavaScript	Node.js · Express (เว็บเซิร์ฟเวอร์/API) · Socket.io (real-time) · express-session (ล็อกอิน) · express-rate-limit (กันเดารหัส) · dotenv (เก็บรหัสลับ)	รับข้อมูลจาก ESP32 รัน Auto Mode (คุม pH, Flood & Drain, เติมน้ำ) บันทึกประวัติ ตรวจสิทธิ์ผู้ใช้ และส่งข้อมูลให้หน้าเว็บทันที
ฐานข้อมูล	SQL	Supabase (PostgreSQL บนคลาวด์ ใช้ฟรี)	เก็บรอบปลูก ค่าที่วัดด้วยมือ ผู้ใช้ และการตั้งค่า Auto Mode ข้อมูลยังอยู่แม้ Server รีสตาร์ท
โครงหน้าเว็บ	HTML	Bootstrap 5 · Font Awesome (ไอคอน)	โครงสร้างหน้า Login และ Dashboard รวมเมนู การ์ด ฟอร์ม และหน้าต่างส่งออก PDF
หน้าตาเว็บ	CSS	ฟอนต์ IBM Plex Sans Thai (Google Fonts)	ธีมสีเขียวแบบกระจกฝ้า (Liquid Glass) มี Dark Mode ใช้ได้ทั้งมือถือและคอม (Responsive) และจัดหน้า A4 ตอนพิมพ์ PDF
การทำงานหน้าเว็บ	JavaScript	Chart.js (กราฟ) · Socket.io client	อัปเดตค่าบนจอแบบ real-time วาดกราฟ ส่งคำสั่ง Relay สร้างตารางสรุปรายวัน และสร้างรายงาน PDF
โฮสต์ / Deploy	บริการ	Render (เซิร์ฟเวอร์คลาวด์) · GitHub (เก็บโค้ด)	ให้เว็บออนไลน์ตลอด 24 ชั่วโมงโดยไม่ต้องเปิดโน้ตบุ๊ก แก้โค้ดแล้ว push ขึ้น GitHub ระบบจะ deploy ให้เอง
เครื่องมือพัฒนา	โปรแกรม	VS Code · Claude Code (AI ผู้ช่วยเขียนโค้ด) · Arduino IDE	เขียนและแก้โค้ดทั้งระบบ แล้วอัปโหลดโค้ดลงบอร์ด ESP32

ไฟล์หลักในโปรเจกต์

esp32/smart_farm.ino C++
โค้ดบนบอร์ด: อ่านเซ็นเซอร์ ส่งข้อมูล คุม Relay และ Failsafe

server.js · routes.js JS
จุดเริ่มต้นของ Server เส้นทาง API ทั้งหมด และระบบล็อกอิน

autoMode.js JS
ตรรกะอัตโนมัติ: คุม pH, Flood & Drain, เติมน้ำ

cropCycles.js · cropStore.js JS
จัดการรอบปลูกแยก 2 ลัง และบันทึกลง Supabase

supabase-setup.sql SQL
คำสั่งสร้างตารางในฐานข้อมูล (รันครั้งเดียว)

dashboard.html · login.html HTML
โครงสร้างหน้าเว็บ

style.css CSS
ธีม สี Dark Mode Responsive และรูปแบบตอนพิมพ์

dashboard.js JS
การทำงานหน้าเว็บทั้งหมด: real-time กราฟ รายงาน PDF

แผนผังหน้าเว็บ: แต่ละเมนูทำอะไร ใช้อะไร

🌱 Smart Farm Web

- 🔐 **หน้า Login** HTML Express-session
  ล็อกอินด้วยชื่อ/รหัสผ่าน (เข้ารหัสด้วย scrypt) แบ่งสิทธิ์ **admin** = ควบคุมได้ทั้งหมด · **viewer** = ดูอย่างเดียว
- 📊 **ภาพรวม** Socket.io
  การ์ดค่าเซ็นเซอร์ real-time: อุณหภูมิ ความชื้น แสง pH 2 ลัง แรงดัน กระแส และสถานะบอร์ด Online/Offline
- 💧 **ระดับน้ำ** JS CSS
  แถบระดับน้ำ 6 ถัง (หน่วย mm) จากเซ็นเซอร์อัลตราโซนิก
- 🎛️ **ควบคุม (admin)** Node.js
  สลับโหมด AUTO / MANUAL และกดเปิด/ปิด Relay 8 ตัวเอง
  - **คุม pH**
    pH สูงเกินค่าที่ตั้ง → ปล่อยน้ำยาลด pH ชั่วครู่ แล้วพัก 5 นาที
  - **Flood & Drain**
    ปล่อยน้ำเข้าลัง → แช่ราก → ระบายออก วนตามชั่วโมงที่ตั้งไว้
  - **เติมน้ำอัตโนมัติ**
    เติมจากถังน้ำเติมเข้าลังปลูกตามระดับที่ตั้งไว้
- 📈 **ประวัติ** Chart.js
  กราฟย้อนหลัง 24 ชั่วโมง กราฟละ 1 ค่า แยกแท็บ อุณหภูมิ · แสง · pH · ไฟฟ้า · ระดับน้ำ
- 🥦 **รอบปลูก / รายงาน** JS Supabase
  เริ่ม/เก็บเกี่ยวแยกกันได้ทีละลัง บันทึกทุก 5 นาทีจนเก็บเกี่ยว
  - **ตารางสรุปรายวัน**
    ค่าต่ำสุด/สูงสุด/เฉลี่ยของแต่ละวัน
  - **ค่าที่วัดด้วยมือ**
    กรอกเองตอนบอร์ดออฟไลน์ ใน PDF มีเครื่องหมาย * กำกับ
  - **ส่งออก PDF** CSS print
    กราฟ + ตารางรายชั่วโมง แยกไฟล์ละลัง ไว้ใส่เล่มโปรเจกต์
- 👥 **จัดการผู้ใช้ (admin)** JS
  เพิ่ม/ลบบัญชีผู้ใช้ และกำหนดสิทธิ์
- 🌙 **Dark / Light Mode** CSS
  สลับธีมได้ทุกหน้า และจำค่าไว้ในเบราว์เซอร์

ลำดับการทำงานของระบบ

1. **เซ็นเซอร์วัดค่า**: ESP32 อ่านอุณหภูมิ ความชื้น แสง pH ไฟฟ้า และระดับน้ำ 6 ถัง C++
2. **ส่งขึ้นอินเทอร์เน็ต**: ทุก 5 วินาที ESP32 ส่งข้อมูล (JSON) ไปที่ Server ผ่าน HTTPS พร้อมรหัสอุปกรณ์ C++
3. **Server ตัดสินใจ**: ในโหมด AUTO จะเช็ค pH และระดับน้ำ แล้วเลือกว่าต้องเปิดปั๊มตัวไหน Node.js
4. **ตอบคำสั่งกลับ**: Server ตอบสถานะ Relay ทั้ง 8 ตัว แล้ว ESP32 เปิด/ปิดปั๊มตามนั้น
5. **แสดงผลทันที**: Server ส่งค่าใหม่ให้ทุกหน้าเว็บที่เปิดอยู่ผ่าน Socket.io ไม่ต้องรีเฟรช Socket.io
6. **บันทึกข้อมูล**: ประวัติบันทึกทุก 1 นาที ส่วนรอบปลูกบันทึกทุก 5 นาทีลง Supabase เพื่อทำกราฟและรายงาน PDF SQL

จุดเด่นของระบบ

Real-time
ค่าบนจอเปลี่ยนทันทีที่บอร์ดส่งข้อมูลมา ไม่ต้องกดรีเฟรช

Failsafe ป้องกันปั๊มค้าง
ถ้าบอร์ดติดต่อ Server ไม่ได้เกิน 30 วินาที จะปิดปั๊มทุกตัวเอง

ใช้งานได้จากทุกที่
เว็บอยู่บนคลาวด์ (Render) เปิดได้จากมือถือ ไม่ต้องเปิดคอมทิ้งไว้

ข้อมูลไม่หาย
เก็บบนฐานข้อมูล Supabase ข้อมูลยังอยู่แม้ Server รีสตาร์ท

ปลอดภัย
มีระบบล็อกอิน แบ่งสิทธิ์ เข้ารหัสรหัสผ่าน ESP32 ต้องใช้ Device Key และจำกัดจำนวนครั้งที่ล็อกอินผิด

พลังงานสะอาด
ใช้ไฟจากโซลาร์เซลล์และแบตเตอรี่ 12V

Responsive + Dark Mode
ใช้ได้ทั้งมือถือและคอมพิวเตอร์ สลับธีมสว่าง/มืดได้

รายงาน PDF อัตโนมัติ
สร้างกราฟและตารางรายชั่วโมงของแต่ละรอบปลูกได้ด้วยปุ่มเดียว

หมายเหตุ: ระบบมีโหมด**ข้อมูลจำลอง**สำหรับกราฟย้อนหลังและรายงาน ทำไว้เพราะเวลาทดลองไม่พอ (อาจารย์ที่ปรึกษาอนุญาต) ส่วนค่าปัจจุบันบนหน้าภาพรวมเป็นค่าจริงจาก ESP32 เสมอ